



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

Glossario

Versione	2.0.0
Stato	Approvato
Redazione	Giulia Barzon, Elisa Marchioro
Verifica	Matteo Cuogo, Elisa Marchioro
Approvazione	Matteo Cuogo
Proprietario	ByteMe
Uso	Interno

Registro dei cambiamenti

Versione	Data	Autore	Descrizione	Verifica
2.0.0	24/04/2026	Matteo Cuogo	Approvazione documento	
1.2.1	22/04/2026	Giulia Barzon	Aggiunti termini relativi ai pattern	Matteo Cuogo
1.2.0	20/04/2026	Giulia Barzon	Aggiunti nuovi termini della fase PB	Matteo Cuogo
1.1.3	20/04/2026	Giulia Barzon	Aggiunti termini stateless e Zustand	Matteo Cuogo
1.1.2	12/04/2026	Giulia Barzon	Aggiunti termini relativi a prompt e LLM	Matteo Cuogo
1.1.1	21/03/2026	Elisa Marchioro	Aggiunta sezione "Script: glossario.py"	Matteo Cuogo, Tommaso Tom-bacco
1.1.0	16/03/2026	Giulia Barzon	Aggiunti termini relativi alle tecnologie del POC	Elisa Marchioro
1.0.0	20/02/2026	Matteo Cuogo	Approvazione documento	
0.3.0	11/01/2026	Elisa Marchioro	Aggiunti termini relativi alle metriche del PdQ	Matteo Cuogo
0.2.0	11/01/2026	Giulia Barzon	Aggiunti termini relativi alle metriche del PdQ	Matteo Cuogo
0.1.8	27/12/2025	Joseph Grant	Aggiunta sezione 'verifica' al versionamento	Elisa Marchioro
0.1.7	15/12/2025	Giulia Barzon	Aggiunte parole relative a Piano di qualifica e testing	Elisa Marchioro
0.1.6	14/12/2025	Giulia Barzon	Aggiunte parole requisiti funzionali, non funzionali, di qualità, di vincolo e Piano di qualifica	Elisa Marchioro
0.1.5	8/12/2025	Elisa Marchioro	Aggiunte parole toolbar, user story, milestone, issue, commit, merge	Matteo Cuogo
0.1.4	5/12/2025	Giulia Barzon	Verifica documento e aggiunte parole relative al Piano di Progetto e ai casi d'uso	Elisa Marchioro
0.1.3	4/12/2025	Elisa Marchioro	Aggiunta parole agile, scrum, varie fasi dello sprint, RTB, PB, CA, verifica, validazione	Matteo Cuogo
0.1.2	28/11/2025	Elisa Marchioro	Piccoli fix al documento	Matteo Cuogo
0.1.1	26/11/2025	Elisa Marchioro	Aggiunta parole analisi requisiti, API, backlog, database, distant writing, efficienza, efficacia, git, github, LLM, markdown, MVP, PoC, repository, six thinking hats, sprint	Matteo Cuogo
0.1.0	18/11/2025	Chiara Grossele	Stesura iniziale del documento	Elisa Marchioro

Tabella 1: Registro delle versioni del documento

Indice

1	Introduzione	1
2	Struttura del documento	1
3	Script: glossario.py	1
A		2
B		3
C		4
D		6
E		7
F		8
G		9
H		10
I		11
J		12
K		13
L		14
M		15
N		16
O		17
P		18
Q		20
R		21
S		23
T		25
U		26
V		27
W		28
X		29
Y		30

1 Introduzione

Il presente documento ha lo scopo di raccogliere i termini utilizzati nella documentazione del gruppo ByteMe per il progetto di Ingegneria del Software nell'anno accademico 2025/2026. L'intento dietro a questo documento è la delucidazione di termini sconosciuti e specializzati del dominio del progetto.

2 Struttura del documento

Il documento è strutturato in sezioni dedicate ad una specifica lettera dell'alfabeto. I termini contenuti in questo documento sono stati appresi dalle lezioni, dalle diapositive del corso di Ingegneria del Software e da uno studio autonomo del gruppo.

3 Script: glossario.py

Lo script glossario.py consente di aggiungere automaticamente la “G” in pedice alle parole presenti nel glossario all'interno di tutti i documenti del repository. In questo modo si automatizza un processo che, se svolto manualmente, risulterebbe lungo e oneroso.

A

- **Actual Cost (AC):** Costo reale sostenuto per il lavoro eseguito su un'attività durante un determinato periodo. Include i costi diretti (es. ore lavorate per il costo orario) necessari per realizzare il pacchetto di lavoro. Metrica definita nel Piano di Qualifica.
- **Aderenza al Ruolo (AR):** Metrica di qualità che valuta quanto l'output di un LLM sia coerente con il tono, lo stile e il comportamento definiti nel *System Prompt*. Viene misurata attraverso una valutazione manuale su scala discreta (0, 1, 2) per verificare la capacità del modello di "rimanere nel personaggio" assegnato.
- **Affidabilità (Reliability):** È la capacità del prodotto software di svolgere le proprie funzioni specificate sotto determinate condizioni e per un determinato periodo di tempo, mantenendo un livello di prestazioni corretto. Metrica tipica: Failure Density (densità di fallimenti nei test).
- **Agile:** È un modo di sviluppare software basato su cicli di lavoro brevi, collaborazione continua e adattamento ai cambiamenti, così da poter migliorare il prodotto passo dopo passo e consegnare rapidamente valore all'utente.
- **Allucinazione (AI):** Fenomeno tipico dei modelli linguistici generativi in cui il sistema produce informazioni fattualmente errate, inventate o non deducibili dal testo di input fornito. La riduzione di tale fenomeno è un obiettivo primario dell'assicurazione della qualità nel progetto Second Brain.
- **Analisi dei requisiti:** Fase in cui si raccolgono e si comprendono i bisogni del cliente e la definizione delle funzionalità che il software deve fornire.
- **Analisi Dinamica:** Attività di verifica che prevede l'esecuzione del codice software (oggetti di prova) su un processore reale o virtuale, fornendo dati in input e controllando che l'output corrisponda ai risultati attesi (oracolo). Corrisponde ai test "veri e propri" (Test di unità, Test di integrazione, Test di sistema, Test di accettazione, Test di Regression).
- **Analisi Statica:** Attività di controllo del software effettuata senza eseguire il codice. Include la revisione dei documenti, l'analisi automatica della sintassi e del rispetto degli standard di codifica tramite strumenti. Appartiene alla categoria dei test non funzionali e aiuta a prevenire difetti prima dell'esecuzione.
- **API:** Insieme di regole che permette a diversi software di comunicare tra loro. È come un "ponte" che permette a un software di usare funzioni o dati di un altro software.
- **Adeguatezza funzionale:** L'adeguatezza funzionale indica se il software svolge correttamente le funzioni per cui è stato progettato. Un sistema di buona qualità include tutte le funzionalità richieste, le esegue nel modo giusto e le rende utili per permettere all'utente di raggiungere i propri obiettivi. Questa caratteristica si verifica confrontando i requisiti iniziali con ciò che è stato realmente implementato e attraverso i test di accettazione.

B

- **Backlog:** Elenco delle cose da fare nel progetto, organizzato in base a priorità.
- **Branch Coverage (Copertura dei Rami):** Metrica specifica di copertura che verifica se ogni singolo ramo (decisione vero/falso) del flusso di controllo è stato attraversato almeno una volta dai test.
- **Budget at Completion (BAC):** Il budget totale preventivato per il completamento dell'intero progetto. Corrisponde alla somma di tutti i budget allocati per i singoli pacchetti di lavoro definiti nel Preventivo. Metrica definita nel Piano di Qualifica.
- **Bulletproof React:** Un pattern architetturale (anche noto come *Feature-based Architecture*) per applicazioni frontend che organizza il codice sorgente raggruppando file e componenti per funzionalità (*feature*) anziché per tipologia (es. componenti, hook, servizi). Questo approccio garantisce alto incapsulamento, basso accoppiamento e maggiore scalabilità orizzontale.

C

- **CA (Customer Acceptance):** Fase in cui il cliente controlla il prodotto consegnato e conferma che rispetta i requisiti richiesti.
- **Checklist:** Lista di controllo contenente elementi specifici da verificare manualmente durante le attività di revisione (Inspection o Walkthrough) di documenti o codice. Serve a garantire che errori comuni (es. ortografia, formattazione, mancanza di caption) siano individuati e corretti in modo sistematico.
- **Code Coverage (Copertura del Codice):** Misura che indica la percentuale di codice sorgente che viene eseguita durante l'esecuzione di una suite di test automatici. Serve a valutare l'efficacia dei test: più è alta, minore è il rischio di difetti non rilevati, anche se il cento per cento non garantisce l'assenza di errori.
- **Commit:** Registrazione di un insieme di modifiche nella repository, accompagnata da un messaggio che descrive cosa è stato modificato.
- **Complessità Ciclomantica (McCabe):** Metrica software che misura la complessità del flusso di controllo di un programma contandone il numero di percorsi linearmente indipendenti. Si calcola con la formula $v(G) = e - n + 2p$ (dove e è il numero di archi, n il numero di nodi, p il numero di componenti connesse). Un valore alto indica un codice difficile da testare e mantenere, poiché cresce con la presenza di branch, salti e iterazioni. Metrica definita nel Piano di Qualifica.
- **Conformità Strutturale (CS):** Metrica di qualità che misura la capacità del modello di rispettare rigorosamente i vincoli atomici e le istruzioni operative fornite nello *User Prompt* (es. richiesta di formato Markdown, assenza di preamboli, lunghezza massima).
- **Context (React Context):** Meccanismo nativo di React per condividere stato e funzionalità tra componenti senza dover passare esplicitamente le props attraverso ogni livello della gerarchia (prop drilling). Nel POC è utilizzato per gestire lo stato globale di autenticazione (*AuthContext*), del documento aperto (*DocNameContext*) e dello storico delle generazioni AI (*HistoryContext*).
- **Copertura dei Requisiti (CR):** Metrica che indica la percentuale di requisiti definiti nell'Analisi dei Requisiti che sono stati effettivamente implementati nel software e verificati tramite test. Metrica definita nel Piano di Qualifica.
- **Cost Performance Index (CPI):** Indice di efficienza dei costi che misura il valore del lavoro completato rispetto al costo effettivo sostenuto. Un valore pari a 1 indica efficienza perfetta; inferiore a 1 indica inefficienza (costi superiori al previsto). Metrica definita nel Piano di Qualifica.
- **Cost Variance (CV):** Differenza tra il valore guadagnato (Earned Value) e il costo effettivo (Actual Cost). Indica se il lavoro svolto è costato più o meno di quanto pianificato. Metrica definita nel Piano di Qualifica.
- **Cruscotto di Qualità:** sezione del Piano di Qualifica (o uno strumento software) che visualizza in modo sintetico lo stato corrente del progetto rispetto agli obiettivi prefissati. Utilizza grafici e tabelle per confrontare i valori misurati (es. costi, tempi, errori) con i valori attesi (ottimali o accettabili). Serve a monitorare l'andamento nel tempo di metriche definite nel Piano di Qualifica.

- **CSS Modules:** Sistema di scope locale per i fogli di stile CSS in cui ogni file `.module.css` genera nomi di classe univoci a tempo di build, evitando conflitti di stile tra componenti diversi. Le classi vengono importate come oggetto JavaScript e applicate tramite `className={styles.nomeClasse}`.

D

- **Database:** Sistema per memorizzare e gestire dati in modo organizzato.
- **Debito Tecnico (Technical Debt):** metafora che indica il costo implicito di una rielaborazione aggiuntiva causata dalla scelta di una soluzione facile e rapida (ma non ottimale) invece di utilizzare un approccio migliore che richiederebbe più tempo. Può essere consapevole (deliberato) o inconsapevole (inavvertito). Se non si "ripaga" il debito (es. facendo refactoring), il costo degli errori residui e della manutenzione cresce nel tempo.
- **Definition of Done (DoD):** Insieme di criteri rigorosi che una funzionalità o un incremento di software deve soddisfare per essere considerati "completati" dal team. Garantisce che il lavoro consegnato sia di alta qualità.
- **Distant Writing:** Modalità di scrittura in cui l'utente fornisce le specifiche e l'LLM genera il testo completo.
- **Docker:** Piattaforma software che permette di eseguire applicazioni in ambienti isolati e riproducibili chiamati container. Ogni container include tutto il necessario per eseguire l'applicazione (codice, dipendenze, configurazione), garantendo coerenza tra ambienti di sviluppo e produzione.
- **Docker Compose:** Strumento che permette di definire e orchestrare applicazioni composte da più container Docker tramite un unico file di configurazione YAML (`docker-compose.yml`). Nel POC gestisce i servizi `frontend`, `backend` e `db` e le loro dipendenze.

E

- **Earned Value (EV):** Valore del lavoro effettivamente eseguito espresso in termini di budget approvato per tale lavoro. Rappresenta quanto "valore" è stato prodotto dal progetto fino a un certo istante. Metrica definita nel Piano di Qualifica.
- **Earned Value Analysis (EVA):** Metodologia di gestione del progetto che integra scopo, tempi e costi per misurare le prestazioni del progetto e prevedere i risultati futuri. Permette di identificare precocemente scostamenti rispetto al piano. Riguarda i processi primari di fornitura, definiti nel Piano di Qualifica.
- **EasyMDE:** Libreria JavaScript che fornisce un editor Markdown con interfaccia grafica, barra degli strumenti e anteprima in tempo reale. Nel POC è utilizzata come componente principale dell'interfaccia di editing testuale.
- **Editor di Testo:** Componente principale dell'interfaccia utente del sistema "Second Brain". Si tratta di un editor online avanzato che supporta la sintassi Markdown, fornendo all'utente un'area di scrittura con anteprima in tempo reale del testo formattato. È l'ambiente in cui l'utente crea, modifica e visualizza i contenuti, nonché il punto di interazione per invocare le funzionalità del Sistema AI sul testo selezionato o completo.
- **Efficacia:** Misura della capacità di raggiungere l'obiettivo prefissato.
- **Efficienza (Efficiency):** L'efficienza riguarda la velocità del software e il modo in cui utilizza le risorse del computer. Un sistema efficiente risponde in tempi brevi, riesce a gestire molti utenti o richieste e non spreca memoria, potenza di calcolo o banda di rete. Nei sistemi di intelligenza artificiale si considera anche il costo necessario per generare ogni risposta.
- **Endpoint:** Punto di accesso specifico di un'API REST, identificato da un URL e da un metodo HTTP (GET, POST, PUT, DELETE). Ogni endpoint espone un'operazione ben definita che il client può invocare per interagire con il server.
- **Estimated at Completion (EAC):** Previsione del costo totale del progetto al momento del suo completamento, basata sulle prestazioni misurate fino alla data corrente. Metrica definita nel Piano di Qualifica.

F

- **Feature-based architecture:** Approccio architetturale che organizza il codice sorgente per dominio applicativo anziché per tipo di file. Ogni funzionalità dell'applicazione risiede in una cartella dedicata contenente i propri componenti, hook, chiamate API e stili, favorendo la coesione e riducendo l'accoppiamento tra moduli distinti.
- **Flask:** Micro-framework Python per lo sviluppo di applicazioni web e API REST. Adotta un approccio minimalista che lascia ampia libertà nella scelta dei componenti, risultando adatto a progetti di dimensioni contenute come il POC.

G

- **Gemma 3:** Famiglia di modelli linguistici open-weights all'avanguardia sviluppati da Google. Nel progetto, la versione Gemma 3:4B viene impiegata per task che richiedono alta precisione analitica e rigore comportamentale, come il metodo dei Sei Cappelli di De Bono.
- **Git:** Sistema di controllo versione che permette di tenere traccia delle modifiche al codice sorgente e di collaborare in modo efficiente.
- **Github:** Piattaforma online basata su Git che consente di ospitare repository, collaborare al progetto e gestire versioni e richieste di modifica.
- **Git Hooks:** Script automatici eseguiti da Git prima o dopo eventi specifici (come commit, push, merge). Nel progetto viene utilizzato in particolare l'hook di **pre-push** per eseguire automaticamente la suite di test unitari e bloccare il caricamento di codice fallato sulla repository remota (Quality Gate).
- **Glossario:** Raccolta di vocaboli meno comuni in quanto limitati a un ambiente o propri di una determinata disciplina, accompagnati ognuno dalla spiegazione del significato o da altre osservazioni. Nel nostro caso raccoglierà tutte le parole relative al nostro progetto che meritano di ulteriori precisazioni o che vengono applicate in un particolare contesto.
- **Ground Truth:** Nel contesto della validazione dell'output dell'IA, rappresenta l'informazione di riferimento "vera" e verificata (il testo originale fornito dall'utente) rispetto alla quale viene misurata l'accuratezza e l'assenza di allucinazioni dell'output generato.

H

- **Hook (React Hook):** Funzione speciale di React che permette di utilizzare stato, ciclo di vita e altri meccanismi di React all'interno di componenti funzionali. I custom hook, per convenzione prefissati con **use**, incapsulano logica riutilizzabile separandola dalla presentazione.

I

- **Indice di Gulpease (IG):** Indice di leggibilità calibrato sulla lingua italiana. Valuta la comprensibilità di un testo basandosi sulla lunghezza delle parole e delle frasi. Un valore più alto indica una maggiore leggibilità. Metrica definita nel Piano di Qualifica.
- **Inspection (Ispezione):** Tecnica di revisione statica formale e strutturata, eseguita da un gruppo di pari con l'obiettivo specifico di rilevare difetti e non conformità. A differenza del Walkthrough, l'Inspection si basa sull'uso di checklist per verificare il rispetto di standard e regole predefinite. Nel Piano di Qualifica l'Inspection è spesso preferita al Walkthrough per le attività di verifica ufficiale perché garantisce un controllo più oggettivo e approfondito grazie alle checklist.
- **Issue:** Attività, richiesta o problema tracciato all'interno di un sistema di gestione del progetto (come GitHub), che richiede attenzione o sviluppo.

J

- **JSX**: Estensione della sintassi JavaScript che permette di scrivere strutture simili all'HTML direttamente nel codice JavaScript. Viene transpilato in chiamate React prima dell'esecuzione ed è utilizzato per definire la struttura visuale dei componenti.

K

L

- **Llama 3.2:** Famiglia di modelli linguistici open-weights sviluppati da Meta. Nel sistema Second Brain, il modello Llama 3.2:3B è utilizzato per operazioni editoriali standard e traduzioni, offrendo un ottimo compromesso tra velocità e capacità espressiva.
- **LLM (Large Language Model):** Modello di intelligenza artificiale addestrato su vasti dataset testuali per comprendere e generare linguaggio naturale. Costituisce il motore computazionale del backend per tutte le operazioni di elaborazione del testo.
- **Lines of Code (LOC):** Metrica che misura la dimensione di un programma software contando il numero di righe di testo nel codice sorgente. Si considerano solo le righe logiche di codice, escludendo commenti e righe vuote. Metrica definita nel Piano di Qualifica, tuttavia non vincolante.

M

- **Manuale Utente:** Documento redatto con l'obiettivo di guidare l'utilizzatore finale nell'interazione con il prodotto software. Fornisce istruzioni operative chiare e accessibili, coprendo le fasi di installazione, configurazione iniziale e l'utilizzo ordinario di tutte le funzionalità implementate. Include inoltre procedure di risoluzione per le problematiche più comuni e sezioni di supporto per facilitare l'autonomia dell'utente durante l'intera esperienza d'uso.
- **Manutenibilità (Maintainability):** La manutenibilità indica quanto è semplice correggere, modificare e migliorare il software nel tempo. Un sistema manutenibile ha un codice ben organizzato, facile da capire e supportato da test, in modo che gli sviluppatori possano aggiungere nuove funzionalità o risolvere problemi senza creare nuovi errori.
- **Markdown:** Linguaggio di markup leggero per la formattazione di testo semplice.
- **MVP (Minimum Viable Product):** Versione minima del prodotto contenente tutte le funzionalità obbligatorie.
- **Milestone:** Punto significativo nel percorso del progetto che segna il completamento di una fase o un risultato importante.
- **Merge:** Operazione che combina le modifiche provenienti da un ramo di sviluppo con un altro, integrando il lavoro dei vari membri del team.

N

- **nginx**: Server web ad alte prestazioni utilizzato nel POC come server statico per servire il bundle React compilato e come reverse proxy per instradare le richieste verso il backend Flask. Viene eseguito all'interno del container **frontend**.
- **Norme di Progetto**: Documento formale che definisce e descrive il Way of Working adottato dal gruppo. Contiene le regole, le convenzioni, i processi e le linee guida che il team si impegna a seguire per l'organizzazione del lavoro, la comunicazione, la gestione della repository, la redazione e verifica della documentazione, l'assegnazione dei ruoli e la pianificazione delle attività. Le Norme di Progetto costituiscono un riferimento condiviso per garantire coerenza, qualità e controllo durante lo svolgimento del progetto, e sono soggette a versionamento e aggiornamento in base all'evoluzione delle esigenze del team e del progetto stesso.

O

- **Orchestratore:** Pattern architetturale in cui un componente "supervisore" coordina il flusso di dati e le interazioni tra diversi moduli o livelli dell'applicazione (come Model, View e ViewModel). Un orchestratore non esegue direttamente logica di business complessa né si occupa dei dettagli del rendering visivo, ma si limita a istanziare i componenti necessari, recuperare i dati dallo stato globale e iniettarli dove servono, fungendo da "regista" del sistema.
- **ORM (Object-Relational Mapping):** Tecnica che permette di interagire con un database relazionale utilizzando oggetti del linguaggio di programmazione al posto di query SQL dirette. Nel POC si utilizza SQLAlchemy come ORM per mappare le tabelle del database PostgreSQL in classi Python.

P

- **Paradigma Reattivo** (Reactive Paradigm): Modello di programmazione orientato alla gestione asincrona dei flussi di dati e alla propagazione automatica dei cambiamenti. Nello sviluppo di interfacce utente moderne (come in React), adottare un paradigma reattivo significa che l'interfaccia grafica (View) si aggiorna automaticamente in risposta alle mutazioni dello stato sottostante (Model o ViewModel), senza che lo sviluppatore debba manipolare manualmente il DOM (Document Object Model). Questo approccio si sposa con la programmazione dichiarativa: lo sviluppatore descrive come l'interfaccia dovrebbe apparire per ogni possibile stato dei dati, e il framework si occupa autonomamente di "reagire" ai cambiamenti, sincronizzando visivamente l'applicazione.
- **PascalCase**: Convenzione di nomenclatura in cui ogni parola inizia con la lettera maiuscola, senza separatori (es. `EditorPage`, `AuthContext`). In React è obbligatoria per i nomi dei componenti: una funzione con iniziale minuscola non viene riconosciuta come componente e causa violazioni delle Rules of Hooks.
- **PB (Product Baseline)**: È la definizione completa del prodotto dal punto di vista funzionale e progettuale, usata come base per sviluppo, test e verifica. Include anche l'MVP (Minimum Viable Product), cioè la versione minima del prodotto che contiene solo le funzionalità essenziali per essere testata dagli utenti e ricevere feedback.
- **PDCA (Plan-Do-Check-Act)**: metodo di gestione iterativo utilizzato per il controllo e il miglioramento continuo dei processi e dei prodotti. Nel Piano di Qualifica rappresenta la strategia di gestione della qualità: Plan (pianificare obiettivi e metriche), Do (eseguire le attività), Check (verificare i risultati tramite il Cruscotto), Act (applicare correzioni se i risultati non sono accettabili).
- **Piano di Progetto (PdP)**: Documento strategico e operativo che definisce il piano di esecuzione complessivo del progetto, descrivendo come il team intende raggiungere gli obiettivi prefissati. Il suo scopo è fornire una roadmap strutturata e condivisa, che indichi cosa deve essere fatto, da chi, con quali risorse, entro quando e quali rischi si possono incontrare. Serve come base per organizzare il lavoro, monitorare lo stato di avanzamento, gestire le deviazioni e comunicare in modo trasparente l'andamento a committenti e proponenti.
- **Piano di Qualifica (PdQ)**: Documento che definisce la strategia, le metodologie e le risorse necessarie per garantire la qualità del prodotto software e del processo di sviluppo. Esso stabilisce gli obiettivi di qualità (tramite metriche e soglie di accettazione), pianifica le attività di verifica e validazione (analisi statica e dinamica) e descrive le procedure di miglioramento continuo (ciclo PDCA). Il PdQ funge da riferimento per assicurare che il progetto soddisfi i requisiti funzionali, di qualità e di vincolo concordati.
- **Planned Value (PV)**: Valore del lavoro pianificato per essere completato entro una certa data. Rappresenta il budget autorizzato assegnato al lavoro che deve essere svolto. Metrica definita nel Piano di Qualifica.
- **PostgreSQL**: Sistema di gestione di database relazionale open source, noto per la sua conformità agli standard SQL, l'affidabilità e il supporto a tipi di dati avanzati. Nel POC è utilizzato per la persistenza di utenti, documenti, storici e generazioni AI.
- **Processi di Supporto**: Insieme di processi che supportano e garantiscono la qualità del prodotto e dei processi primari lungo tutto il ciclo di vita del software (es. Documentazione, Gestione della Qualità, Gestione della Configurazione).

- **Processi Primari di Fornitura:** Processi che riguardano le attività e i compiti del fornitore (il gruppo di progetto) necessari per preparare e consegnare il prodotto software al committente, inclusa la gestione manageriale ed economica.
- **Processi Primari di Sviluppo:** Processi che includono le attività di realizzazione tecnica del prodotto software, dall'analisi dei requisiti alla progettazione, codifica e integrazione.
- **Prompt:** Insieme di istruzioni testuali inviate a un modello generativo per guidarne la risposta. Un prompt ben progettato (Prompt Engineering) è essenziale per ottenere output di alta qualità e minimizzare le risposte errate.
- **Proof of Concept (PoC):** Prototipo iniziale e limitato del software, sviluppato per dimostrare la fattibilità delle funzionalità core e dell'idea del progetto in generale.
- **Prop drilling:** Pratica di sviluppo tipica di React (e di altri framework basati su componenti) che consiste nel passare dati o funzioni (le props) da un componente padre a un componente figlio situato molto più in basso nell'albero, attraversando svariati componenti intermedi che non utilizzano direttamente quei dati. Sebbene sia utile per mantenere un flusso di dati unidirezionale e prevedibile, un prop drilling eccessivo può rendere il codice difficile da leggere e mantenere.
- **Pytest:** Un framework di testing per il linguaggio Python, caratterizzato da una sintassi semplice e dalla scalabilità per suite di test complesse. Viene utilizzato nel progetto per la verifica unitaria dei moduli e delle funzioni del backend.

Q

- **Quality Gates (Cancelli di Qualità):** Criteri di controllo automatico imposti al codice per verificare se la qualità rispetta gli standard minimi definiti. Permettono di stabilire se il progetto può passare a uno stadio successivo o se una modifica può essere accettata. Se il codice non supera il *Quality Gate*, la modifica viene bloccata.
- **Qualità dell'output generativo:** La qualità dell'output generativo, tipica dei sistemi basati su intelligenza artificiale come il nostro, riguarda quanto le risposte o i contenuti prodotti dal sistema sono corretti, coerenti e utili. Un buon sistema generativo fornisce informazioni accurate, evita di inventare dati e risponde in modo pertinente alle richieste dell'utente, mantenendo anche una certa coerenza tra risposte simili. Questa qualità viene valutata sia con misure automatiche sia attraverso il giudizio di persone che analizzano i risultati.

R

- **React:** Libreria JavaScript open source sviluppata da Meta per la costruzione di interfacce utente basate su componenti riutilizzabili. Adotta un modello dichiarativo in cui l'interfaccia è descritta in funzione dello stato, e aggiorna automaticamente il DOM al variare di quest'ultimo.
- **React Router:** Libreria standard per la gestione della navigazione client-side nelle applicazioni React. Permette di definire route associate a componenti e di navigare tra pagine senza ricaricare l'intera applicazione. Nel POC è utilizzata nella versione 6.
- **React Testing Library (RTL):** Una libreria per il testing di componenti React che si concentra sul testare il comportamento dell'applicazione dal punto di vista dell'utente, interagendo con i nodi del DOM anziché testare i dettagli implementativi interni dei componenti.
- **Repository:** Spazio in cui viene salvato il codice del progetto insieme a tutte le versioni e i file utili. Permette al team di lavorare insieme senza perdere il lavoro svolto.
- **Requirements Stability Index (RSI):** Metrica che quantifica la volatilità dei requisiti del progetto. Misura la percentuale di requisiti che non hanno subito modifiche in un determinato periodo rispetto al totale dei requisiti. Metrica definita nel Piano di Qualifica.
- **Requisiti funzionali:** Descrivono i comportamenti del sistema. Sono le azioni che il software deve compiere in risposta a determinati input. Se un requisito funzionale non è soddisfatto, manca una funzionalità. Rispondono alla domanda: *Cosa deve fare il software?*
- **Requisiti non funzionali:** I requisiti non funzionali descrivono le proprietà del sistema o i vincoli entro cui deve operare. Non aggiungono "cose da fare", ma stabiliscono quanto bene il sistema deve farle. Rispondono alla domanda: *Come si deve comportare il sistema mentre fa le cose?*
- **Requisiti di qualità:** I requisiti di qualità definiscono quanto bene il sistema deve funzionare. Non riguardano cosa fa il sistema, ma le sue caratteristiche intrinseche come affidabilità, manutenibilità, efficienza e usabilità. Spesso derivano da standard ISO (come ISO/IEC 25010). Rispondono alla domanda: *Quali standard di eccellenza deve rispettare il mio codice/prodotto?*
- **Requisiti di vincolo:** I vincoli sono limitazioni imposte dall'esterno (dal committente, dalla legge, o dalla tecnologia disponibile) entro le quali dobbiamo muoverci. Non abbiamo libertà di scelta su questi aspetti, sono i "paletti" del progetto. Esempio requisiti di vincolo:
 - Tecnologici: il progetto deve essere un'applicazione web basata su HTML/JS e che l'editor deve essere basato su Markdown.
 - Architetture: l'applicazione deve funzionare prevalentemente client-side (sul browser).
 - Interfaccia esterna: usare le API standard (tipo OpenAI) per collegarsi all'LLM. Licenza/Consegna: il codice deve essere pubblicato su un repository pubblico (es. GitHub) con licenza open-source.
- **REST (Representational State Transfer):** Stile architetturale per la progettazione di API web basato su risorse identificate da URL e su operazioni espresse tramite metodi HTTP standard (GET, POST, PUT, DELETE). Le API REST sono stateless: ogni richiesta contiene tutte le informazioni necessarie per essere elaborata indipendentemente.

- **RTB (Requirements and Technology Baseline):** Punto di riferimento formale e congelato del progetto, rappresentato da un documento approvato che stabilisce in modo definitivo l'insieme dei requisiti funzionali e non funzionali del sistema, insieme alle scelte architetturali e tecnologiche fondamentali su cui si baserà lo sviluppo.

S

- **Separation of Concerns:** Principio di progettazione software secondo cui ogni modulo o componente deve avere una responsabilità ben definita e separata dalle altre. Nel POC si applica separando le chiamate HTTP (livello `api/`), la logica applicativa (livello `hooks/`) e la presentazione (livello `components/`).
- **Sessione (Flask Session):** Meccanismo lato server per mantenere lo stato di autenticazione dell'utente tra richieste HTTP successive. Nel POC Flask utilizza cookie firmati con la `FLASK_SECRET_KEY` per associare ogni richiesta alla sessione dell'utente autenticato.
- **Schedule Performance Index (SPI):** Indice di efficienza della schedulazione che misura la velocità di avanzamento del progetto rispetto a quanto pianificato. Un valore inferiore a 1 indica ritardo sulla tabella di marcia. Metrica definita nel Piano di Qualifica.
- **Schedule Variance (SV):** Differenza tra il valore guadagnato (Earned Value) e il valore pianificato (Planned Value). Indica se il progetto è in anticipo o in ritardo rispetto alla schedulazione. Metrica definita nel Piano di Qualifica.
- **Scrum:** È un metodo Agile che organizza il lavoro in Sprint regolari e utilizza ruoli ed eventi definiti per aiutare il team a lavorare in modo ordinato e trasparente.
- **Sistema AI:** Componente software del sistema "Second Brain" che integra un Large Language Model (LLM) per fornire funzionalità di elaborazione intelligente del testo. È un attore secondario che opera in modo trasparente all'utente, abilitando operazioni come: riassunto, riscrittura, traduzione, correzione grammaticale, analisi critica tramite il metodo dei "Sei Cappelli per Pensare", e generazione di testo automatico (distant writing). Il sistema AI interagisce con il testo selezionato o completo inserito nell'editor.
- **Sistema di Persistenza:** Componente architetturale del sistema "Second Brain" responsabile della memorizzazione, del recupero e della gestione dei dati degli utenti. È un attore secondario che opera a livello locale, cioè gestisce il salvataggio e il caricamento di file direttamente sul dispositivo dell'utente; a livello cloud (opzionale per utenti autenticati), cioè gestisce un archivio centralizzato su database server-side, abilitando operazioni CRUD (Create, Read, Update, Delete) remote e accesso sincronizzato ai documenti.
- **Six Thinking Hats (Sei Cappelli per Pensare):** Metodologia di Edward De Bono per l'analisi critica da diverse prospettive.
- **Sprint:** Periodo di lavoro breve e programmato (2 settimane nel nostro caso) in cui il team realizza un insieme di attività. Durante lo sprint i membri del gruppo cambiano ruolo per favorire collaborazione e crescita delle competenze.
- **Specifica Tecnica:** Documento di carattere tecnico-progettuale rivolto ai membri del team di sviluppo e ai revisori, volto a descrivere l'architettura di alto e basso livello del sistema. Formalizza le scelte relative ai design pattern, alla scomposizione dei componenti software, alla modellazione dei dati per la persistenza e alle logiche di integrazione con servizi o API esterne. La sua finalità è fornire una base conoscitiva rigorosa per garantire la manutenibilità, la scalabilità e la corretta implementazione tecnica del software.
- **Sprint Planning:** Riunione che apre lo Sprint, in cui il team decide quali attività svolgere e come affrontarle.
- **Sprint Execution:** La fase dello Sprint in cui il team lavora concretamente sulle attività pianificate, sviluppando e testando le funzionalità.

- **Sprint Review:** Incontro alla fine dello Sprint dove il team mostra il lavoro completato e raccoglie feedback da Product Owner e stakeholder.
- **Sprint Retrospective:** Riunione conclusiva dello Sprint in cui il team riflette sul proprio modo di lavorare e propone miglioramenti per lo Sprint successivo.
- **SQLAlchemy:** Libreria Python che fornisce strumenti ORM e un livello di astrazione per l'accesso al database. Permette di definire modelli come classi Python e di eseguire operazioni sul database tramite un'API orientata agli oggetti, gestendo automaticamente connessioni e transazioni.
- **Stateless** (Senza stato): In architettura del software, si riferisce a un sistema, server o protocollo di comunicazione in cui non viene memorizzata alcuna informazione relativa allo stato della sessione o alle interazioni precedenti del client. In un'architettura stateless ogni singola richiesta HTTP inviata al server deve essere autocontenuta, ovvero deve includere tutte le informazioni necessarie affinché il server possa comprenderla ed elaborarla in modo totalmente indipendente. Nel contesto del progetto Second Brain, il backend è progettato come gateway stateless: non memorizza documenti, login o cronologie, ma si limita ad eseguire elaborazioni on-demand (come le chiamate LLM), delegando interamente la persistenza e la gestione dello stato al frontend.
- **System Prompt:** Parte di un prompt che definisce l'identità, il ruolo, i vincoli etici e il tono di voce che l'intelligenza artificiale deve assumere durante l'interazione. Non è visibile direttamente dall'utente finale ma ne condiziona profondamente il comportamento.

T

- **Tasso di Allucinazione (HR):** Indice statistico che esprime la frequenza relativa di generazioni contenenti allucinazioni rispetto al numero totale di campioni analizzati durante le sessioni di test.
- **Toolbar:** Elemento dell'interfaccia grafica che raccoglie pulsanti e funzioni di uso frequente, permettendo un accesso rapido alle operazioni principali.
- **Tecnica 0/100:** Tecnica di misurazione dell'avanzamento fisico di un'attività. Prevede che il valore guadagnato (Earned Value) venga assegnato solo al completamento totale dell'attività (100%), mentre un'attività iniziata ma non finita ha valore nullo (0%). Serve a evitare stime soggettive parziali.
- **Temperatura:** Parametro di configurazione degli LLM che controlla il grado di casualità e creatività dell'output. Valori bassi (es. 0.1) rendono il modello deterministico e analitico, mentre valori alti (es. 0.8) ne aumentano la varietà creativa.
- **Test di Accettazione (Acceptance Testing / Collaudo):** Validazione finale per accertare il soddisfacimento dei requisiti utente, svolta nella fase finale prima del rilascio.
- **Test di Integrazione:** Verificano le interfacce e l'interazione tra moduli o sottosistemi già testati singolarmente. Vengono fatti dopo i test di unità, risale il V-Model verso l'architettura (Global Design). Rilevano difetti nella progettazione architetturale o nello scambio dati tra componenti.
- **Test di Regressione:** Non è una fase separata, ma una pratica trasversale. Ogni volta che si modifica il codice o si aggiunge una funzionalità, si eseguono di nuovo i test precedenti per assicurarsi di non aver rotto funzionalità già esistenti.
- **Test di Sistema:** Verificano il comportamento dell'intero sistema software completo rispetto alle specifiche tecniche. Verificano il sistema completo rispetto ai requisiti funzionali e non funzionali. Includono test non funzionali (performance, sicurezza, usabilità). Comprendono anche smoke test (test rapidi di stabilità). Vengono fatti dopo l'integrazione, corrisponde alla verifica dei "System Requirements".
- **Test di Unità (Unit Testing):** È la verifica del più piccolo componente software funzionante autonomamente (es. un metodo o una classe in Java). Vengono fatti durante la codifica (Coding) o addirittura prima (TDD - Test Driven Development). Corrispondono alla fase di "Detailed Design" nel V-Model. Devono essere Automatici, Thorough (accurati/esaustivi), Ripetibili, Indipendenti (l'ordine non conta) e Professionali (codice di qualità).

U

- **Usabilità (Usability):** L'usabilità descrive quanto il software è facile da imparare e da usare. Un buon sistema è intuitivo, permette agli utenti di svolgere i propri compiti senza difficoltà e riduce la possibilità di commettere errori. Questa qualità si misura osservando come le persone reali utilizzano il software e raccogliendo le loro opinioni.
- **User Prompt:** La componente di un prompt che contiene il comando specifico, i task operativi e il testo di input inserito dall'utente per una specifica elaborazione.
- **User story:** Descrizione sintetica di una funzionalità dal punto di vista dell'utente, utilizzata nei metodi Agile per chiarire cosa il sistema deve permettere di fare.
- **Utente Autenticato:** Categoria di utente che ha completato il processo di registrazione e/o accesso al sistema. Questo profilo abilita l'accesso a funzionalità avanzate del prodotto, tra cui la persistenza cloud dei propri documenti, la gestione remota (CRUD) dei file e l'accesso persistente e sincronizzato alle note senza operazioni manuali di upload/download. È un attore primario nel sistema, destinato a utenti che necessitano di un ambiente di lavoro persistente e personalizzato.
- **Utente Non Autenticato:** Categoria di utente che interagisce con il sistema senza effettuare registrazione o login. Ha accesso alle funzionalità base del prodotto, tra cui l'utilizzo dell'editor Markdown, le operazioni AI su testo (riassunto, riscrittura, correzione, etc.) e la persistenza locale dei file. È un attore primario nel sistema, pensato per un utilizzo immediato, occasionale o privo dell'esigenza di salvataggio remoto e sincronizzazione.

V

- **Validazione:** Processo che verifica se il prodotto risponde ai bisogni reali dell'utente e quindi se è il prodotto giusto da realizzare.
- **Verifica:** Processo che controlla se il prodotto è stato realizzato correttamente, cioè secondo specifiche e progettazione.
- **Versionamento:** Pratica sistematica di assegnare identificativi univoci e incrementali alle diverse versioni di un documento o di un componente software, al fine di tracciarne l'evoluzione nel tempo; garantisce tracciabilità, consente il controllo delle modifiche e facilita il ripristino di versioni precedenti.
- **Vite:** Strumento di build moderno per applicazioni JavaScript che offre un server di sviluppo con ricaricamento istantaneo e una pipeline di compilazione ottimizzata basata su Rollup. Nel POC è utilizzato come bundler e dev server per il frontend React.
- **Vitest:** Un framework di testing nativo per Vite, progettato per essere estremamente veloce. Nel progetto viene utilizzato per l'esecuzione dei test unitari e di integrazione dell'ambiente frontend (React).
- **V-Model (Modello a V):** modello di sviluppo software sequenziale che evidenzia la relazione diretta "uno a uno" tra le fasi di sviluppo (lato sinistro della V) e le corrispondenti fasi di test (lato destro della V). Serve a capire che non si testa "alla fine", ma ogni fase di test verifica una specifica fase di progettazione.

W

- **Walkthrough:** Tecnica di revisione statica (generalmente informale) in cui l'autore di un artefatto software (come un documento di requisiti o una porzione di codice) guida i membri del team attraverso il suo contenuto per ricevere feedback, favorire la comprensione comune e individuare errori evidenti. È utile per la condivisione della conoscenza all'interno del team, ma è meno rigoroso dell'Inspection.
- **Way of Working:** Insieme strutturato di regole, processi, strumenti e pratiche adottate da un gruppo di lavoro per organizzare, coordinare e gestire le attività di un progetto. Definisce le modalità di comunicazione interna ed esterna, la gestione del codice e della documentazione, l'assegnazione e rotazione dei ruoli, nonché i flussi di verifica e approvazione. Il Way of Working ha lo scopo di garantire efficienza, tracciabilità, collaborazione e miglioramento continuo durante l'intero ciclo di vita del progetto.

X

Y

Z

- **Zustand:** Una libreria open-source minimale e ad alte prestazioni per la gestione dello stato globale (state management) in applicazioni React. A differenza di soluzioni più complesse come Redux, Zustand utilizza un approccio semplificato basato sui Custom Hooks, evitando l'uso di boilerplate code. Nel progetto Second Brain, Zustand implementa il livello Model dell'architettura MVVM, fungendo da singola fonte di verità (Single Source of Truth) per i metadati dell'editor e lo storico delle generazioni AI. Inoltre, grazie al suo middleware integrato (**persist**), garantisce la sincronizzazione automatica dello stato globale con il LocalStorage del browser, assicurando la persistenza dei dati senza l'ausilio di un database esterno.